

Algoritmos y Estructuras de Datos 3

Cuestionario del 2do. Examen Parcial

3 de Junio de 2022, Periodo II - 2021

- Observación 1.** Todas las respuestas de este ejercitario tienen el objetivo de que el estudiante pueda aprender de sus respuestas. Por lo tanto, todas las respuestas están escritas en detalles y haciendo énfasis en la explicación. No se pretende que un estudiante durante un examen escriba con el detalle presentado en este escrito, pero, si que pueda expresar sus ideas de forma clara.
- Observación 2.** Este notebook fue elaborado con SageMath 9.2.

1. Caballero generalizado

Tenemos un grilla rectangular de tamaño $F \times C$, donde F es el número de filas y C es el número de columnas. En esta grilla tenemos un caballero generalizado que, dado (M, N) , puede moverse M celdas horizontalmente y N celdas verticalmente en un solo movimiento; esto es, si el caballero está en la celda (a, b) , entonces puede moverse a una celda (c, d) si y solo si $|a - c| = M$ y $|b - d| = N$ o $|a - c| = N$ y $|b - d| = M$. Una condición adicional es que algunas celdas son *celdas prohibidas* indicando que un caballero nunca puede moverse a una celda si esta celda está marcada como prohibida.

Por cada celda c , debes de contar el número de celdas distintas que pueden llegar a c utilizando un solo movimiento del caballero generalizado. Si el número de celdas que alcanzan c es par, entonces esa celda se marca como *par*, y si es impar se marca como *impar*. Una celda prohibida es considerada como par. Debes de escribir un algoritmo que retorne el número de celdas pares e impares.

1. Modela el problema como un grafo. Presenta una definición formal del grafo escribiendo por comprensión o extensión los conjuntos de vértices y aristas. ¿Cuál es el grado mínimo y máximo del grafo?
2. Dibuja en ejemplo pequeño para un tablero de tamaño 3×3 , $M = 2$ y $N = 1$ y 2 celdas prohibidas. ¿Cuántas celdas pares e impares tiene tu ejemplo?
3. Escribe un algoritmo basado en BFS o DFS que recorra el grafo definido y retorne el número de celdas impares y pares. Presenta una explicación del funcionamiento de tu algoritmo.
4. Escribe la complejidad temporal de tu algoritmo en términos de F , C , M y N . Presenta también una justificación de esa complejidad.

Rúbrica

Criterio	1 Punto	2 Puntos	3 Puntos
Grafo	Define formalmente y correctamente el conjunto de vértices o aristas	Define formalmente el conjunto de vértices y aristas	Además de lo anterior, escribe correctamente el grado máximo y mínimo
Dibujo del grafo	Presenta un dibujo correcto basado en la definición formal	Además de lo anterior, presenta una cuenta correcta de las celdas pares e impares	
Algoritmo	Presenta un pseudocódigo que retorna el resultado correcto en la mayoría de las entradas	Presenta un pseudocódigo que retorna el resultado correcto en todas las entradas	
Explicación	Escribe una explicación del algoritmo presentado justificando su funcionamiento		

La complejidad asintótica es

La complejidad asintótica es correcta y presenta una

Solución del Ejercicio 1

Comenzamos construyendo el grafo solicitado. El conjunto de vértices son las celdas del tablero y las aristas son los pares de celdas que son alcanzable con un solo movimiento del caballo. Sean $[F] = \{0, 1, \dots, F - 1\}$ y $[C] = \{0, 1, \dots, C - 1\}$, entonces el grafo $G = (V, E)$ está definido por

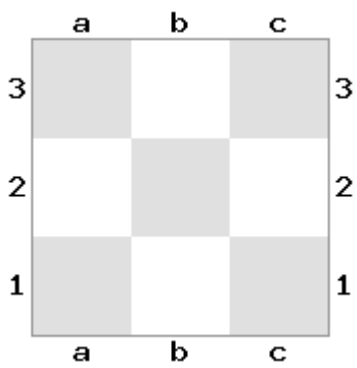
$$V = \{(x, y) \in [F] \times [C] : (x, y) \text{ no es una celda prohibida}\}$$

y

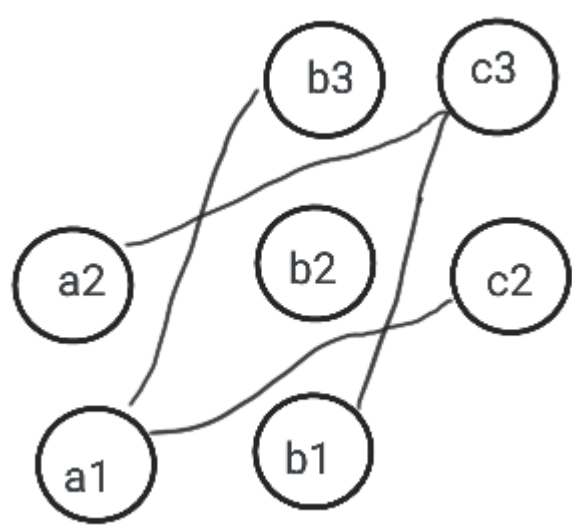
$$E = \{(a, b)(c, d) \in V \times V : |a - c| = M \text{ y } |b - d| = N \text{ o } |a - c| = N \text{ y } |b - d| = M\}.$$

El grado máximo que un vértice puede tener es 8, y el grado mínimo que puede tener es 0.

Ahora consideremos el siguiente tablero de ajedres de tamaño 3×3 .



Supongamos que a3 y c1 son celdas marcadas como prohibidas. Así obtenemos el siguiente grafo.



Del dibujo se observa que hay 3 vértices con grado par y 4 vértices de grado impar.

La estrategia de conteo es realizar un recorrido en DFS y durante el recorrido mantener un contador por cada vértice del número de vecinos. A final, mirar estos contadores para saber el número de vértices con grado impar y par.

```
//DFS sobre el grafo G
CABALLERO-GENERALIZADO(G=(V,E))
begin
  Inicializar vector visitados a 0
  Inicializar vector vecinos a 0
  for u in V(G) do
    if visitados[u] == 0 then
      VISITAR(G,v)//subrutina EXPLORE de Dasgupta et al.
    end-if
  end-for
```

```
    Contar la cantidad de números pares e impares en el vector
    de vecinos y retornar
end
```

```
//se marcan los vértices visitados
//se cuentan los vecinos
EXPLORE(G=(V,E),v)
begin
    visitados[v] = 1
    for vu in E(G) do
        vecinos[v] = vecinos[v]+1
        if visitados[u] == 0 then
            EXPLORE(G,u)
        end-if
    end-for
end
```

La complejidad del DFS es lineal en el número de vértices y aristas, y por lo tanto, el número total de vértices es $O(F \cdot C)$. Como el grado máximo para cualquier tablero es 8, tenemos que el número de aristas es también $O(F \cdot C)$. Notar que el número de aristas es independiente de M y N . Así, la complejidad del algoritmo es $O(F \cdot C)$.

2. Conexiones en el campus

Se están construyendo muchos nuevos edificios en el campus de la UNA y un programador debe de encargarse de diseñar la red de interconexión entre todos los edificios del campus.



Cada edificio se identifica con una coordenada (x, y) en el plano. Cada cable de comunicación conecta exactamente dos edificios siguiendo una línea recta entre los dos edificios y la información viaja en el cable en ambas direcciones. Los cables se pueden cruzar, pero solo conectan los edificios que se encuentran en sus extremos.

Actualmente conocemos el mapa del campus y algunos edificios ya están interconectados. Estos cables existentes de interconexión no deben de ser alterados. Debes de terminar de conectar todos los edificios del campus de tal forma que siempre exista un camino de interconexión entre cualquier par de edificios. Por supuesto, la universidad quiere que utilices la cantidad mínima de cable para la red.

1. Modela el problema como un problema de grafos. ¿Qué sería los vértices y que serían las

- aristas? Una explicación informal es suficiente.
2. Escribe un algoritmo que resuelva el problema. ¿Cuál es el input y el output del algoritmo? Presenta una explicación del funcionamiento de tu algoritmo.
3. Escribe la complejidad asintótica de tu algoritmo en términos del número de vértices y

Rúbrica

Criterio	1 Punto	2 Puntos	3 Puntos
Grafo	Define claramente el conjunto de vértices o aristas	Define formalmente el conjunto de vértices y aristas	Además de lo anterior, presenta una interpretación natural de los vértices y aristas en el contexto del problema
Algoritmo	Escribe explícitamente y correctamente el input y el output	Además de lo anterior, presenta un pseudocódigo que retorna el resultado correcto en la mayoría de las entradas	Además de lo anterior, presenta un pseudocódigo que retorna el resultado correcto en todas las entradas
Explicación	Escribe una explicación del algoritmo presentado justificando su funcionamiento		
Complejidad	La complejidad asintótica es correcta	Además de lo anterior, presenta una justificación correcta de la complejidad	

Total 8 puntos

Solución del Ejercicio 2

Comenzamos construyendo el grafo solicitado. El conjunto de vértices son las coordenadas de los edificios en el plano cartesiano y las aristas son las interconexiones existentes entre los edificios. Entonces el grafo $G = (V, E)$ está definido por

$$V = \{(x, y) \in \mathbb{R}^2 : (x, y) \text{ es la posición de un edificio del campus}\}$$

y

$$E = \{uv : \text{los edificios } u \text{ y } v \text{ están interconectados}\}.$$

El problema consiste en encontrar un subgrafo de expansión mínimo. Si utilizamos el algoritmo de Kruskal, la red de interconexión existe es el conjunto X de aristas ya seleccionadas. Entonces, lo único que queda por hacer es completar el subgrafo inducido por X con nuevas aristas que hagan que X sea un árbol y estas aristas sean lo más barato posible. El pseudocódigo de abajo es una modificación del algoritmo de Kruskal para este problema.

```
//se utilizan las subrutina de Union-Find
KRUSKAL-MODIFICADO(G)
begin
  for u in V(G) do
    MAKESET(u)
  end-for
  //guardamos todas las aristas de G en X
  for uv in E(G) do
    X.add(uv)//el conjunto X guarda las interconexiones
actuales
    UNION(u,v)
  end-for
  //completamos el grafo con las aristas faltantes y
calculamos la distancia entre ellas
  for u in V(G) do
    for v in V(G) do
      if uv not in X then
        Calcular distancia euclidiana entre u y v
        Agregar la arista uv a G
      end-if
    end-for
  end-for
end-for
```

```
Y = []//inicializamos un conjunto Y como vacío
Ordenar E(V) de menor a mayor
for uv in E(V) en orden do
    if uv not in X then//agregar la arista solo si no está
        todavía en X
            if FIND(u) != FIND(v) then
                X.add(uv)
                Y.add(uv)//el conjunto Y guarda solo las aristas
                nuevas que fueron agregadas
                UNION(u,v)
            end-if
        end-if
    end-for
Return Y
end
```

En el algoritmo de arriba el input es el grafo que representa la interconexión actual de los edificios, y el output es el conjunto de aristas nuevas que se agregaron a la red que está representada por el conjunto Y.

La complejidad del algoritmo de Kruskal es $O(n \log m)$, donde n es el número de vértices y m es el número de aristas. Sin embargo, en el pseudocódigo de arriba se agregó un bucle que calcula la distancia euclidiana entre los vértices que no aparecen en X. Como ese bucle se hace por cada par de vértices, entonces, solo ese bucle hace que la complejidad del algoritmo sea $O(n^2)$.

3. Sumas de números primos

Un número entero positivo puede escribirse como una suma de números primos diferentes. Dados dos enteros positivos n y k , tu tarea consiste en contar el número de formas de escribir n como la suma de k números primos diferentes.

Por ejemplo, si $n = 8$, las sumas $3 + 5$ y $5 + 3$ se cuentan una sola vez. Si $n = 24$ y $k = 3$, entonces la respuesta debe de ser 2 porque los conjuntos $\{2, 3, 19\}$ y $\{2, 5, 17\}$ tienen suma igual a 24 y no hay otros conjuntos que sumen 24. Para $n = 24$ y $k = 2$ la respuesta es 3, porque los conjuntos $\{5, 19\}$, $\{7, 17\}$ y $\{11, 13\}$ suman 24 y no hay más conjuntos de tamaño 2. Para $n = 1$ y $k = 1$ la respuesta es 0.

1. Escribe una formulación de programación dinámica que calcule el número de formas de escribir n como la suma de k números primos diferentes. Identifica los casos bases y la formulación inductiva.
2. Escribe la tabla de memoización para $n = 8$ y $k = 2$.
3. Escribe un algoritmo basado en tu formulación que tome n y k como entrada y retorne el resultado. Presenta también una explicación del funcionamiento de tu algoritmo.
4. Realiza un análisis de la complejidad temporal de tu algoritmo del ítem 3.

Rúbrica

Criterio	1 Punto	2 Puntos	3 Puntos
Formulación	Define un estado apropiado para el problema	Construye el paso inductivo correcto a partir del estado definido o identifica correctamente el caso base	Construye el paso inductivo correcto a partir del estado definido e identifica correctamente el caso base
Tabla de memoización	La tabla solicitada es correcta		
Algoritmo	El algoritmo presentado es correcto en algunas entradas y está basado en el formulación DP	El algoritmo presentado es correcto en todas las entradas y está basado en la formulación DP	

Criterio	1 Punto	2 Puntos	3 Puntos
Explicación	Escribe una explicación del algoritmo presentado justificando su funcionamiento		

Solución del Ejercicio 3

Recordemos que los factores primos de n son números comprendidos entre 1 y n , pero que no son 1 o n . Entonces, podemos definir el estado $S(i, n, k)$ como el número de formas de escribir n como la suma de k primos utilizando los primeros i números primos. Así, la formulación DP del problema es

$$S(i, n, k) = S(i', n, k) + S(i', n - i, k - 1),$$

donde i es un número primo e i' es el número primo más grande menor a i . Lo que dice esta formulación es que el i -ésimo número primopuede o no contribuir a la suma. Si contribuye a la suma, decrementamos k en 1 y n en i ; en caso contrario mantenemos k y resolvemos de forma recursiva.

Los casos bases son los siguientes. Para $k \leq 0$ tenemos que $S(i, n, k) = 0$, porque ningún número puede escribir con 0 términos. Para $i \leq 1$ tenemos que $S(i, n, k) = 0$, porque 1 y 0 no son números primos. También, para $n \leq 1$ tenemos $S(i, n, k) = 0$. Si $k = 1$, entonces $S(i, n, k) = 1$ si y solo si n es primo y $n \leq i$; en caso contrario es 0.

La tabla de memoización para los casos $k = 1$ y $k = 2$ se muestran abajo. Se omiten las filas y columnas para los casos $i \leq 1$ y $n \leq 1$ donde como vimos los valores son todos 0s. Las filas corresponden a la variable i y las columnas a la variable n .

k=1	1	2	3	4	5	6	7	8
2	0	1	0	0	0	0	0	0
3	0	1	1	0	0	0	0	0
5	0	1	1	0	1	0	0	0
7	0	1	1	0	1	0	1	0

k=2	1	2	3	4	5	6	7	8
2	0	0	0	0	0	0	0	0
3	0	0	0	0	1	0	0	0
5	0	0	0	0	1	0	1	1
7	0	0	0	0	0	0	1	1

Para construir el algoritmo utilizamos un enfoque bottom-up. La idea es la de llenar la tabla de arriba para cualquier n y k que se reciben como parámetros y aplicando de forma iterativa la formulación DP. Llenamos la tabla primero comenzando con los casos bases, luego por cada valor que puede tomar k , siempre llenando primero las filas de arriba hasta abajo y las columnas de izquierda a derecha.

```
SUMA-PRIMOS(n,k)
begin
    //se puede calcular estos primos, por ejemplo, con la criba
    de Eratóstenes en tiempo O(n^1/2)
    Sea m la cantidad de números primos menores a n
    Crear un arreglo S[1..m][1..n][1..k]//el primer índice son
    los primos calculados en el paso anterior
    //inicializamos los casos bases para k=1
    for i primo menor a n do
        for j=1 to n do
            if j==i then
                S[i][j][1] = 1
            else-if
                S[i][j][1] = 0
            end-if
        end-for
    end-for
```

```

        end-for
    end-for
    //aquí aplicamos la recursión cuidando los casos bases
    for l=1 to k do
        for i primo menor a n do
            for j=1 to n do
                //r es la variable i' en la formulación
                //estos condicionales cuidan los casos bases
                if r<=1 then
                    S[i][j][l] = 0
                else-if j<=1 then
                    S[i][j][l] = 0
                else
                    S[i][j][l] = S[r][j][l]+S[r][j-r][l-1]
                end-if
            end-for
        end-for
    end-for
    Return S[i][n][k]//i es el primo más grande menor a n
end

```

El cálculo de los números primos es $O(\sqrt{n})$ por la criba de Eratóstenes en el primer paso. La inicialización de los casos bases tiene un costo de $O(mn)$. Luego, tenemos que recorrer toda la tabla para llenar y esto tiene un costo de $O(mnk)$. Como el algoritmo de Eratóstenes produce a lo sumo $O(\sqrt{n})$ números primos, tenemos que el costo total es $O(kn^{3/2})$. Notar que esta complejidad depende mucho de que algoritmo utilizamos para calcular los números primos. Podemos utilizar un algoritmo de búsqueda exhaustiva en lugar de la criba de Eratóstenes y obtendríamos así un costo total de $O(kn^3)$.